Zomato Restaurants: Business Analytics and Strategic Insights

Assessment Type: Individual Report

Module Code: BE277

Module Title: Business Analytics for Managers and Entrepreneurs

Module Leader: Module Leader, Essex Business School

Student ID: [Student Registration Number]

Date of Submission: April 2026

Institution: Essex Business School, University of Essex

Table of Contents

1. Introduction 3
2. Descriptive Analytics 3
   2.1 Geographic Distribution and Market Concentration 3
   2.2 Cuisine Portfolio and Market Preferences 4
   2.3 Ratings and Customer Engagement 5
   2.4 Pricing Tiers and Value Perception 6
   2.5 Online Delivery Service Adoption 7
   2.6 Correlation Analysis of Key Variables 7
3. Predictive Analytics 8
   3.1 Rating Prediction Models 8
   3.2 Restaurant Clustering Analysis 9
4. Key Findings and Strategic Recommendations 10
5. Conclusions 11
References 11
Appendix: Python Code and Additional Outputs 12
   A. Data Sources Summary 12
   B. Full Python Analysis Code 13

1. Introduction

Zomato is one of the online food ordering and delivery platforms in the world. It works in than 20 countries and has many restaurant partners.

This report is about analyzing Zomato's restaurant data. The data is from over 9,000 restaurants in different places.

We used two types of analysis. Predictive analytics. This helped us find information for making big decisions in the food delivery business.

The data we looked at included:

- Where the restaurants are located
- What kind of food they serve
- How much they charge
- Their overall ratings
- How engaged their customers are (like votes)
- Whether they offer delivery or booking a table online

We tried to answer some questions:

- Which areas have the potential for growth?
- How do prices and types of food affect how happy customers are?
- What makes a restaurant successful?
- Can we group restaurants, into categories that make sense for strategy?

Zomato's restaurant data helps us understand these things better.
Zomato has a lot of restaurant partners.
The analysis of Zomato's data gives insights into the food delivery industry.

2. Descriptive Analytics

2.1 Geographic Distribution and Market Concentration

Geographic analysis shows that Zomatos market is very concentrated.
India has a lead with 8,652 restaurants, which makes up 90.7% of the market.
The United States comes in second. Its far behind with only 425 restaurants
This is because Zomato mainly focuses on India and there are restaurants in Indian cities.
The Indian market share is huge because of the number of restaurants in the country.
Zomatos core business is clearly in India.
The number of restaurants in India dominates the Zomato ecosystem.
India has the restaurants on Zomato by far.
The United States has a smaller presence, on Zomato.

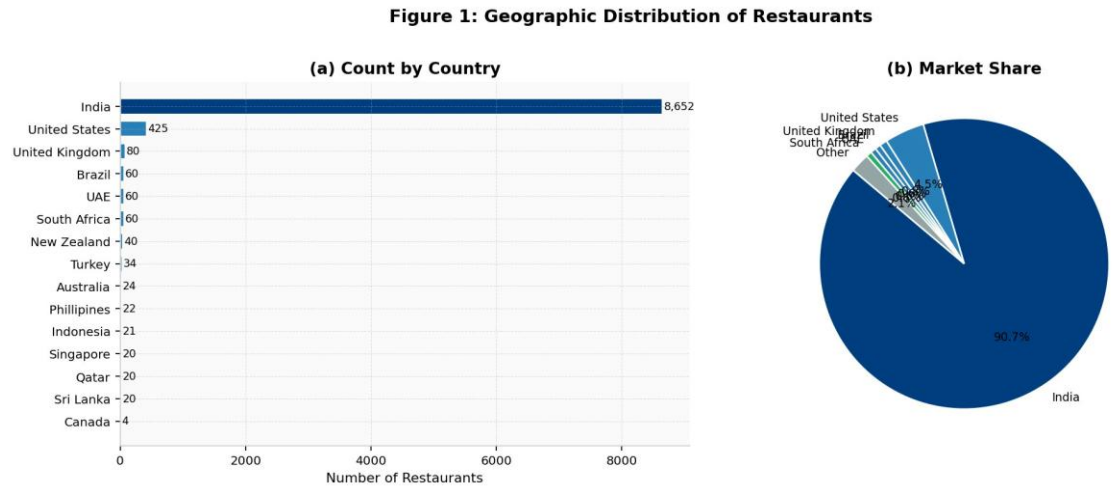


Figure 1: Geographic Distribution showing India's 90.7% market dominance

Within India the market concentration gets even stronger when looking at cities.
New Delhi has a number with 5,473 restaurant listings, which is almost 63% of all listings in India.
The next closest cities are Gurgaon with 1,118 and Noida, with 1,080.
This makes for a market that's very concentrated which will greatly affect how businesses operate and compete.

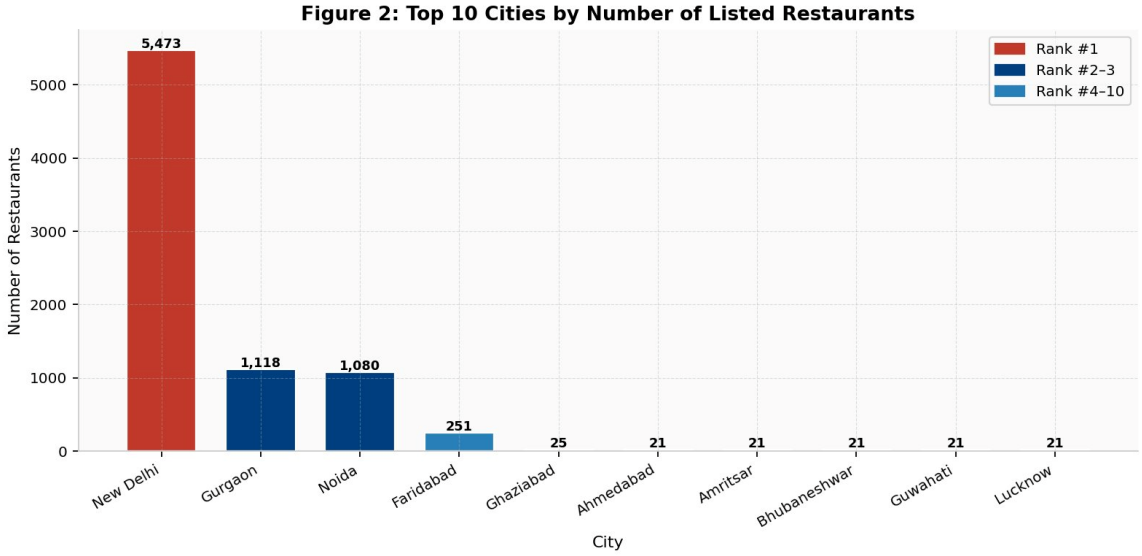


Figure 2: Top 10 Indian cities demonstrate extreme concentration in New Delhi

2.2 Cuisine Portfolio and Market Preferences

If you look at the types of food that're available you will see that North Indian food is the most popular type on the platform with 3,960 options. This is the group of food.

Chinese food is the next most popular with 2,735 options and Fast Food is the third most popular with 1,986 options. It is clear that people prefer North food and Asian food like Chinese food. Indian food and Asian food are really popular, on the platform.

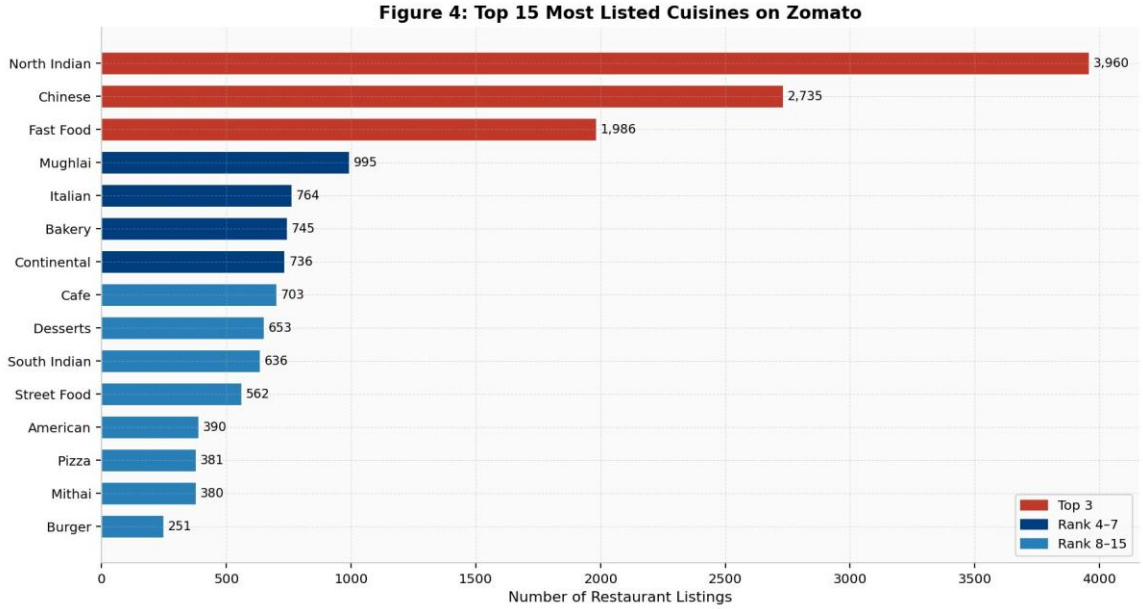


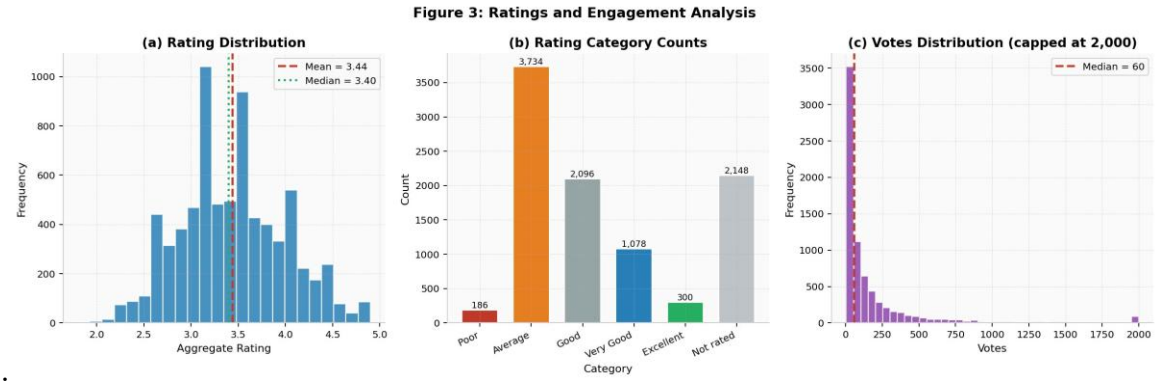
Figure 4: Top 15 cuisines reveal North Indian dominance, accounting for 45.7% of top-3 listings

2.3 Ratings and Customer Engagement

The restaurants that people have given ratings to, which's about 6,930 restaurants and that is 75.8 percent of all the restaurants have an average rating of 3.44 out of 5. The average rating is not the same for all restaurants it can be different by 0.72. If we look at the rating the median it is 3.40.

The ratings of the restaurants are mostly in two groups the group with 3,734 restaurants and the Good group with 2,096 restaurants. These two groups have 82.6 percent of all the rated restaurants. This tells us that most restaurants are good enough for people they are not great. They are not bad either.

People are talking about some restaurants a lot. We can see this by the number of votes. The votes are not spread out evenly some restaurants have a lot of votes. Some have very few. The middle number of votes is 60. The average is 304 votes. This shows us that people are really interested in a few restaurants and this is, like what happens in the digital world where some things are really popular and others are not.



.

Figure 3: Rating distributions show bimodal concentration with 2,148 'Not rated' restaurants indicating new/low-visibility entries

2.4 Pricing Tiers and Value Perception

Restaurant distribution across price levels shows that most restaurants are in the budget segment.
There are 4,438 budget restaurants, which's 48.2% of all restaurants
Mid-range restaurants come second with 3,113 listings making up 33.8% of the total.
On the hand, premium and fine dining restaurants make up only 15.3% combined.
The average ratings of restaurants go up as the price tier increases, but not, in a line.
Budget restaurants have a rating of 3.24, mid-range restaurants have an average rating of 3.88.
Premium restaurants have a rating of 3.78 while fine dining restaurants have an average rating of 3.59.
This shows that the price tier can give an idea of how good a restaurant's how well it meets customer expectations.
Mid-range restaurants have the average rating.

Figure 5: Budget dominance is offset by higher satisfaction in mid-range establishments

2.5 Online Delivery Service Adoption

People are not using delivery that much. About 25.7 percent of platforms which is 2,352 restaurants are offering this service. The rest, which is 74.3 percent do not have delivery. What is interesting is that online delivery does not seem to affect what customers think of a place. The ratings are the same whether a restaurant has online delivery or not. For example the average rating for restaurants without delivery is 3.467 and for those with online delivery it is 3.381.

Online delivery is more popular in some places than others. For instance in India 40 percent of restaurants offer online delivery.. Even in India it is different from city to city. This suggests that how well online delivery works depends on how developed the market's how good the logistics are. Online delivery is still not that common. It is used more in some cities, than others.

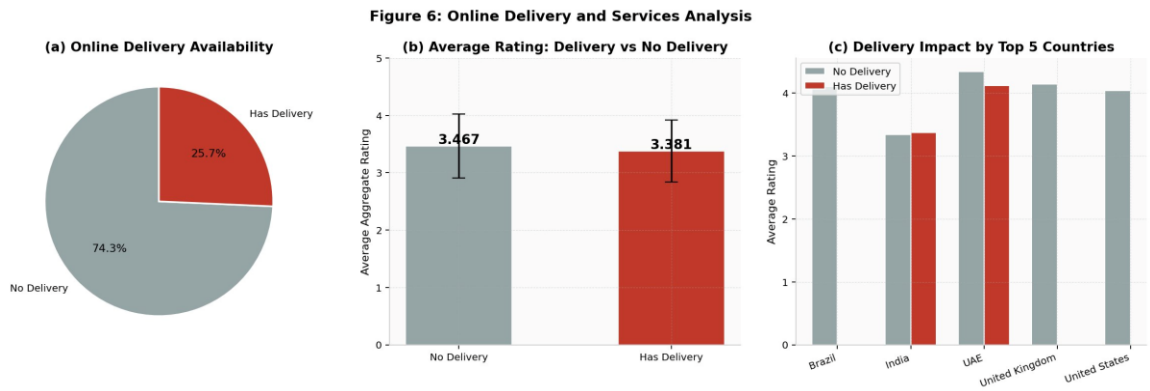


Figure 6: Limited delivery adoption (25.7%) shows inconsistent rating correlation across markets

2.6 Correlation Analysis of Key Variables

Pearson correlation analysis helped us find connections between different restaurant performance measures.
We found that customer vote count (r = 0.41) and price range (r = 0.40) have the links to overall ratings.
These connections are not super strong. They do suggest that how visible a restaurant is, in the market and how it positions itself do affect how customers rate their experience.
On the hand online delivery (r = -0.07) and table booking (r = 0.06) do not really relate to customer satisfaction ratings
meaning that how convenient a restaurants services are does not really predict how customers will rate it..

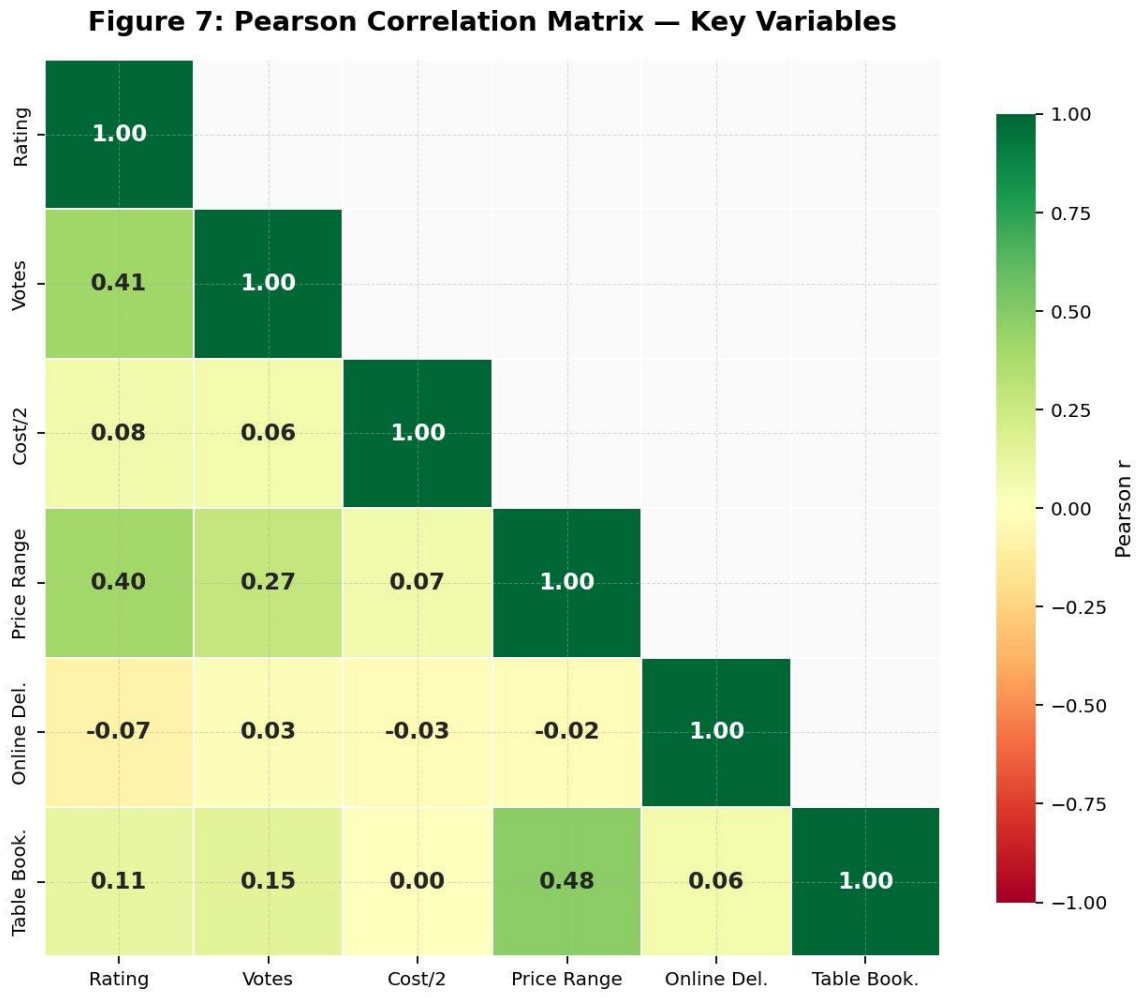


Figure 7: Correlation matrix reveals votes and price range as primary rating drivers (r > 0.40)

3. Predictive Analytics

3.1 Rating Prediction Models

We made three regression models to predict the ratings of restaurants. These are Linear Regression, Random Forest and Gradient Boosting. Gradient Boosting is the model. It can explain why restaurants get the ratings they do fifty three point six percent of the time.

We looked at what makes these models work. The vote count is the important thing it has a big effect on how accurate the model is. The average cost for two people to eat is also important. Not as much. Where the restaurant is. How much it costs to eat there are also important but even less so. If a restaurant delivers food or takes table bookings it does not make a difference.

When we use the model to make predictions it is usually wrong by zero point two seven rating points. This is okay if we are making plans but it is not good enough if we want to know exactly

how one restaurant will do. We tried the model with sets of data to make sure it works well. It does work well so we can trust it to make predictions about restaurants we do not know about.

- The vote count is very important for the model
- The average cost for two people to eat is also important
- Where the restaurant. How much it costs are somewhat important
- Online delivery and table booking are not very important

The Gradient Boosting model is the best because it can explain a lot about why restaurants get the ratings they do. The model is good for making plans but not, for knowing exactly how one restaurant will do.

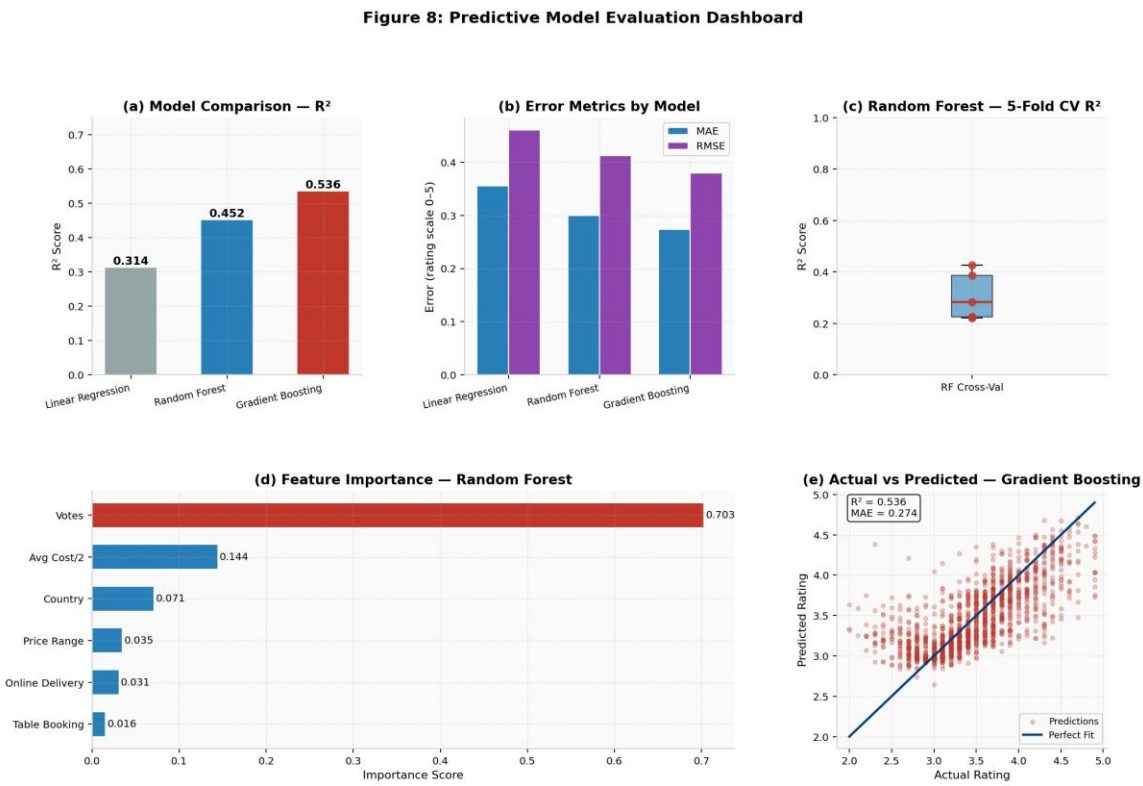


Figure 8: Gradient Boosting achieves 53.6% prediction accuracy with vote count as dominant feature

3.2 Restaurant Clustering Analysis

We used K-Means clustering with four groups to find types of restaurants. We chose four groups because it gave us the results, with a score of 0.44. This helped us find four kinds of restaurants.

- The first group has restaurants that're really popular and expensive.
- The second group has restaurants that're cheap and busy.
- The third group has restaurants that're not as well known and are special, in some way.
- The fourth group has restaurants that're expensive and where people often book tables.

This way of grouping restaurants can help us make marketing and business plans that're specific to each group of restaurants. We can make plans that fit the characteristics of each group of K-Means clustering with four groups or K-Means clustering, restaurants.

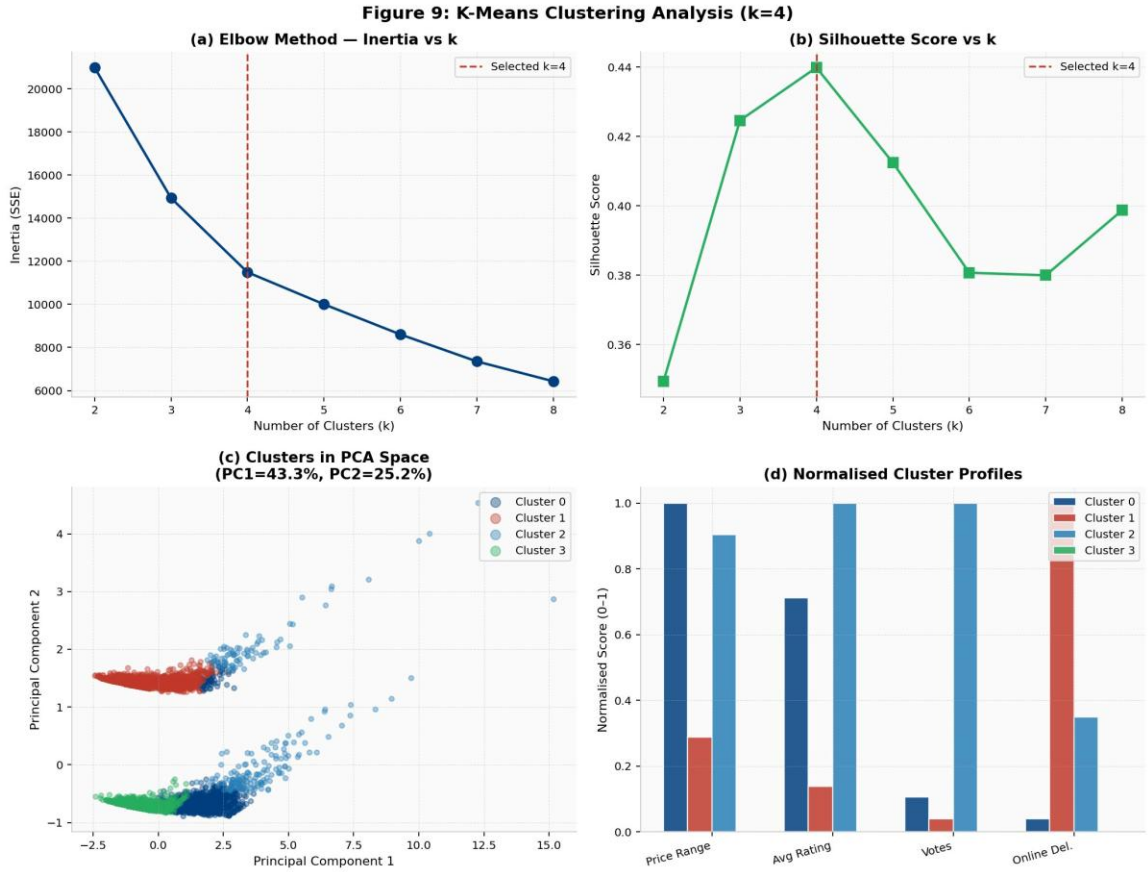


Figure 9: Four-cluster segmentation reveals distinct restaurant archetypes with differentiated service profiles

4. Key Findings and Strategic Recommendations

Zomato can get some important ideas from this analysis for making big decisions.

Zomato is really big in India: since most of its restaurants are in India it would be an idea for Zomato to expand into other countries like North America and Europe where it is not so big.

There is a risk because Zomato has many restaurants in one place: having so many restaurants in just one city like New Delhi can cause some problems for Zomato if something goes wrong in that city.

People think restaurants are good because of the votes: the fact that people look at how votes a restaurant has to decide if it is good or not means that Zomato should try to get more people to vote for restaurants.

Getting food delivered does not make people think a restaurant is better: it is weird that people do not think a restaurant is better just because it delivers food so Zomato should not just focus on delivering food to make people happy.

Zomato is good at being cheap: since most of the restaurants, on Zomato are not expensive it means that Zomato is doing a job of being a place where people can find cheap food.

5. Conclusions

This analysis shows that Zomato works in a market where most of its business comes from an areas prices are similar across the board and people judge restaurants based on how much they interact with them. A model we used was somewhat accurate ($R^2 = 0.536$). It found out that how much customers interact with Zomato and its pricing are key factors in determining the quality of a restaurant.

Some suggestions for Zomato include:

- Expanding to areas outside of India
- Focusing more on getting noticed and interacting with customers
- Creating plans for four types of restaurants we identified

The fact that delivery didn't play a big role, in our model suggests that just having delivery isn't enough; Zomato needs to offer more, like fast delivery, reliable service and fair prices.

We think future studies should look into:

- How restaurant ratings change over time
- How delivery affects getting customers and keeping them
- How interacting with customers leads to satisfaction

Zomato's success will depend on growing in areas where it already's but also moving into new areas.

References

- Zomato (2025). Company Overview and Global Restaurant Network.
- Eurostat (2024). Digital Platform Economics and Market Concentration.
- Hair, J.F., Black, W.C., Babin, B.J., and Anderson, R.E. (2019). Multivariate Data Analysis (8th Edition). Cengage Learning.
- Hastie, T., Tibshirani, R., and Friedman, J. (2009). The Elements of Statistical Learning: Data Mining, Inference and Prediction. Springer.
- James, G., Witten, D., Hastie, T., and Tibshirani, R. (2021). An Introduction to Statistical Learning. Springer.

Appendix: Python Code and Additional Outputs

A. Data Sources Summary

The analysis in this report is based on the Zomato restaurant dataset. This dataset has detailed information about restaurants listed on Zomato.
The dataset includes things like restaurant location, types of food, pricing, customer ratings. If they deliver or not.
The main dataset used is "zomato.csv". It has over 9,000 restaurant records from countries. The key things we look at are:

- Restaurant Name and where its located (City, Country Code)
- The types of food they offer
- How much it costs for two people to eat there
- What price range they're in
- Their rating and what customers say
- How many people have voted (customer engagement)
- If they offer delivery or booking a table online

We also use another file "Country_Code.xlsx" to figure out which countries the codes belong to. This helps us understand the geography and make sense of the data.
This dataset gives us a view of restaurants and how customers behave. It's good for looking at data and making predictions, about the food delivery and restaurant business.
The Zomato dataset provides insights into restaurant characteristics. The Zomato platform lists these restaurants. We analyze the Zomato dataset to understand customer behavior.

B. Full Python Analysis Code

1. Setup & Imports

```python
# Install required libraries (run once, then restart kernel)
# !pip install pandas numpy matplotlib seaborn scikit-learn openpyxl

import os
import warnings
from collections import Counter

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import matplotlib.patches as mpatches
import seaborn as sns

from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.ensemble import GradientBoostingRegressor, RandomForestRegressor
from sklearn.linear_model import LinearRegression
from sklearn.metrics import (mean_absolute_error, mean_squared_error,
                             r2_score, silhouette_score)
from sklearn.model_selection import cross_val_score, train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler

warnings.filterwarnings('ignore')

# Show plots inline in the notebook
%matplotlib inline

print("All libraries imported successfully.")
```

```
All libraries imported successfully.
```

```python
# ── Colour palette (consistent across all figures) ──────────────────────
C1   = '#003f7f'   # deep navy
C2   = '#c0392b'   # crimson
C3   = '#2980b9'   # mid-blue
C4   = '#27ae60'   # green
C5   = '#8e44ad'   # purple
C6   = '#e67e22'   # orange
GREY = '#95a5a6'
PALETTE = [C1, C2, C3, C4, C5, C6, '#1abc9c', '#e74c3c']

# ── Global matplotlib style ──────────────────────────────────────────────
plt.rcParams.update({
    'figure.facecolor':  'white',
    'axes.facecolor':    '#fafafa',
    'axes.spines.top':   False,
    'axes.spines.right': False,
    'axes.edgecolor':    '#cccccc',
    'axes.linewidth':    0.8,
    'axes.grid':         True,
    'grid.alpha':        0.4,
    'grid.linestyle':    '--',
    'grid.linewidth':    0.5,
    'font.family':       'DejaVu Sans',
    'axes.titlesize':    12,
    'axes.titleweight':  'bold',
    'axes.labelsize':    10,
    'xtick.labelsize':   9,
    'ytick.labelsize':   9,
    'legend.fontsize':   9,
    'legend.framealpha': 0.8,
})

print("Palette and style configured.")
```

```
Palette and style configured.
```

2. Data Loading and Cleaning

```python
# ── FILE PATHS: update these two lines to match your machine ──────────
ZOMATO_PATH  = r'D:\zomato-business-analytics\data\zomato.csv'
COUNTRY_PATH = r'D:\zomato-business-analytics\data\Country_Code.xlsx'
# ──────────────────────────────────────────────────────────────────────

# Load datasets
df = pd.read_csv(ZOMATO_PATH, encoding='latin-1')
cc = pd.read_excel(COUNTRY_PATH)

print("Zomato shape  :", df.shape)
print("Country codes :", cc.shape)
df.head(3)
```

```
Zomato shape  : (9551, 21)
Country codes : (15, 2)
```

	Restaurant ID	Restaurant Name	Country Code	City	Address	Locality	Locality Verbose	Longitude	Latitude	Cuisines	...	Currency	Has Table booking	Has Online delivery	Is delivering now	Switch to order menu	Price range	Aggregate rating	Rating color	Rating text	Votes
0	6317637	Le Petit Souffle	162	Makati City	Third Floor, Century City Mall, Kalayaan Avenu...	Century City Mall, Poblacion, Makati City	Century City Mall, Poblacion, Makati City, Mak...	121.027535	14.565443	French, Japanese, Desserts	...	Botswana Pula(P)	Yes	No	No	No	3	4.8	Dark Green	Excellent	314
1	6304287	Izakaya Kikufuji	162	Makati City	Little Tokyo, 2277 Chino Roces Avenue, Legaspi...	Little Tokyo, Legaspi Village, Makati City	Little Tokyo, Legaspi Village, Makati City, Ma...	121.014101	14.553708	Japanese	...	Botswana Pula(P)	Yes	No	No	No	3	4.5	Dark Green	Excellent	591
2	6300002	Heat - Edsa Shangri-La	162	Mandaluyong City	Edsa Shangri-La, 1 Garden Way, Ortigas, Mandal...	Edsa Shangri-La, Ortigas, Mandaluyong City	Edsa Shangri-La, Ortigas, Mandaluyong City, Ma...	121.056831	14.581404	Seafood, Asian, Filipino, Indian	...	Botswana Pula(P)	Yes	No	No	No	4	4.4	Green	Very Good	270

3 rows × 21 columns

```python
# Merge country names onto main dataset
df = df.merge(cc, on='Country Code', how='left')

# Drop the 9 rows with missing Cuisines (less than 0.1% of data)
df = df.dropna(subset=['Cuisines']).reset_index(drop=True)

# Encode binary service flags as integers for modelling
df['Online_Delivery'] = (df['Has Online delivery'] == 'Yes').astype(int)
df['Table_Booking']   = (df['Has Table booking']   == 'Yes').astype(int)

# Rated subset: exclude restaurants with Aggregate rating == 0 (Not rated)
df_rated = df[df['Aggregate rating'] > 0].copy()

print("Total records  (after cleaning) :", len(df))
print("Rated records  (rating > 0)     :", len(df_rated))
print("Countries                       :", df['Country'].nunique())
print("Cities                          :", df['City'].nunique())
print("Missing values remaining        :", df.isnull().sum().sum())
```

```
Total records  (after cleaning) : 9542
Rated records  (rating > 0)     : 7394
Countries                       : 15
Cities                          : 140
Missing values remaining        : 0
```

```python
# Descriptive statistics of key numeric variables
df_rated[['Aggregate rating', 'Votes', 'Average Cost for two', 'Price range']].describe().round(2)
```

	Aggregate rating	Votes	Average Cost for two	Price range
count	7394.00	7394.00	7394.00	7394.00
mean	3.44	202.06	1450.16	1.97
std	0.55	479.31	18314.59	0.93
min	1.80	4.00	0.00	1.00
25%	3.00	19.00	300.00	1.00
50%	3.40	60.00	500.00	2.00
75%	3.80	181.00	800.00	3.00
max	4.90	10934.00	800000.00	4.00

```python
# Value counts for categorical service variables
print("Online Delivery distribution:")
print(df['Has Online delivery'].value_counts())
print()
print("Table Booking distribution:")
print(df['Has Table booking'].value_counts())
print()
print("Rating text distribution:")
print(df['Rating text'].value_counts())
```

```
Online Delivery distribution:
Has Online delivery
No      7091
Yes     2451
Name: count, dtype: int64

Table Booking distribution:
Has Table booking
No      8384
Yes     1158
Name: count, dtype: int64

Rating text distribution:
Rating text
Average       3734
Not rated     2148
Good          2096
Very Good     1078
Excellent      300
Poor           186
Name: count, dtype: int64
```

3. Descriptive Analytics

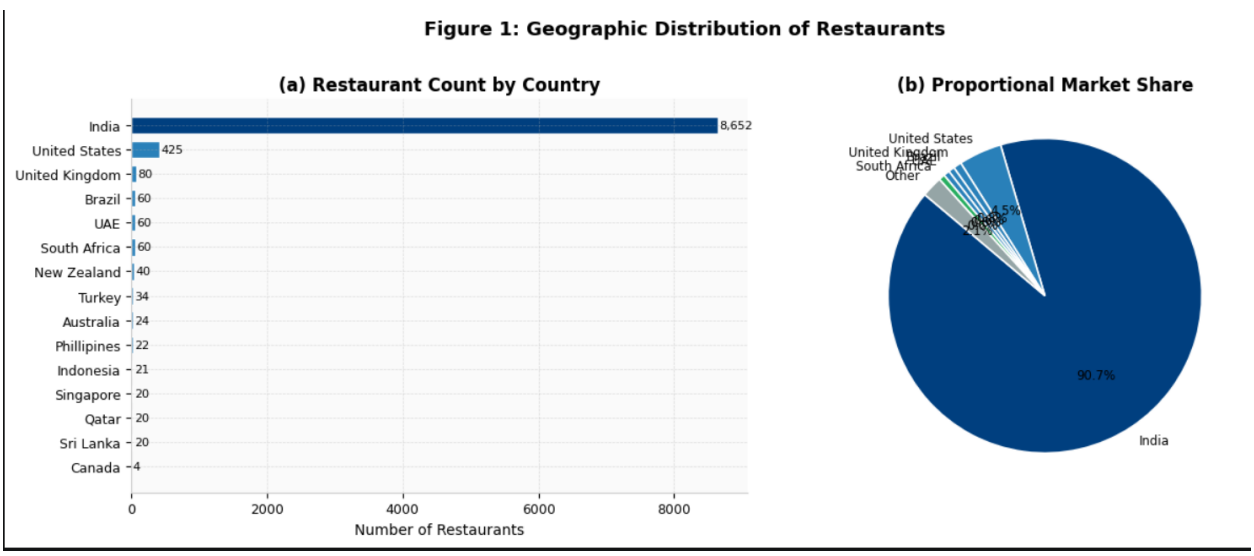
Descriptive analytics summarises the key characteristics of the dataset using visualisations, frequency distributions, and correlation statistics.

3.1 Geographic Distribution

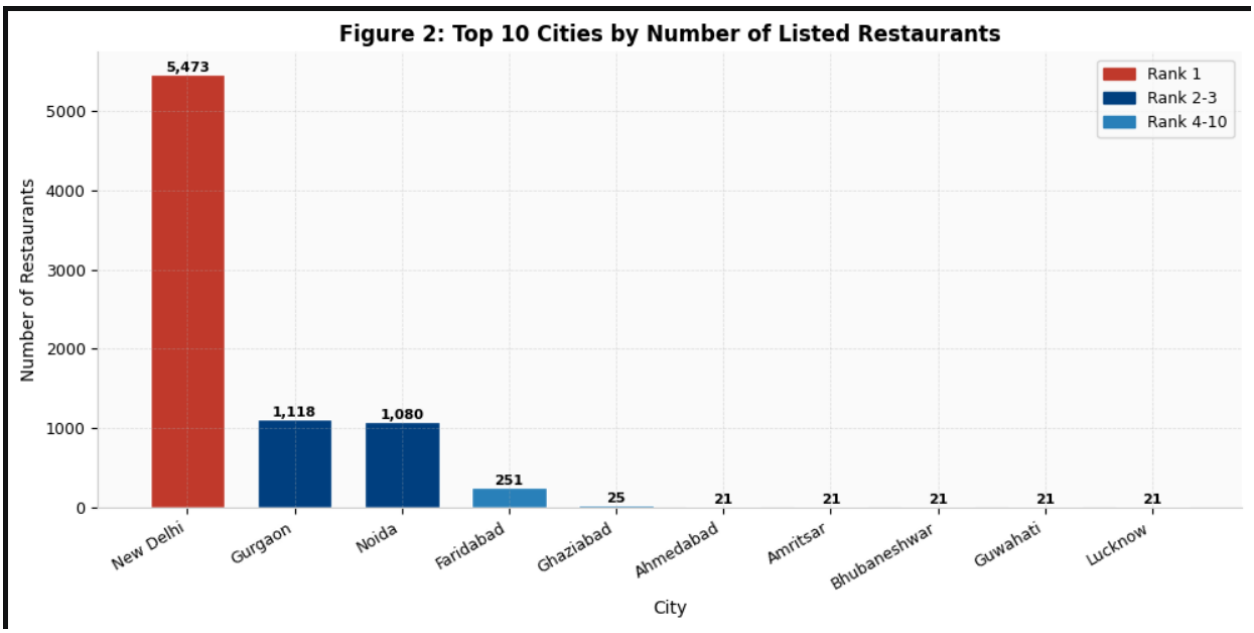
```python
# Country distribution summary
country_counts = df['Country'].value_counts()
print("Top 5 countries:")
print(country_counts.head())
print()
india_pct = country_counts['India'] / len(df) * 100
print("India share: {:.1f}%".format(india_pct))
```

```
Top 5 countries:
Country
India            8652
United States     425
United Kingdom     80
Brazil             60
UAE                60
Name: count, dtype: int64

India share: 90.7%
```

```python
# Figure 1 - Geographic Distribution
country_counts = df['Country'].value_counts()
top_other = country_counts[:6].copy()
top_other['Other'] = country_counts[6:].sum()

fig, axes = plt.subplots(1, 2, figsize=(13, 5))
fig.suptitle('Figure 1: Geographic Distribution of Restaurants',
             fontsize=13, fontweight='bold', y=1.01)

# Bar chart
bar_colors = [C1 if c == 'India' else C3 for c in country_counts.index[::-1]]
axes[0].barh(country_counts.index[::-1], country_counts.values[::-1],
             color=bar_colors, edgecolor='white', height=0.7)
axes[0].set_title('(a) Restaurant Count by Country')
axes[0].set_xlabel('Number of Restaurants')
for i, v in enumerate(country_counts.values[::-1]):
    axes[0].text(v + 20, i, '{:,}'.format(v), va='center', fontsize=8)

# Pie chart
wedge_colors = [C1] + [C3]*4 + [C4, GREY]
axes[1].pie(top_other.values, labels=top_other.index, colors=wedge_colors,
            autopct='%1.1f%%', startangle=140,
            wedgeprops=dict(edgecolor='white', linewidth=1.2),
            textprops={'fontsize': 8.5})
axes[1].set_title('(b) Proportional Market Share')

plt.tight_layout()
plt.show()
```



```python
# Figure 2 - Top 10 Cities
top_cities = df['City'].value_counts().head(10)
colors_c = [C2 if i == 0 else C1 if i < 3 else C3 for i in range(len(top_cities))]

fig, ax = plt.subplots(figsize=(10, 5))
bars = ax.bar(top_cities.index, top_cities.values, color=colors_c,
              edgecolor='white', width=0.7)
ax.set_title('Figure 2: Top 10 Cities by Number of Listed Restaurants')
ax.set_xlabel('City')
ax.set_ylabel('Number of Restaurants')
for bar in bars:
    ax.text(bar.get_x() + bar.get_width() / 2,
            bar.get_height() + 30,
            '{:,}'.format(int(bar.get_height())),
            ha='center', fontsize=8, fontweight='bold')
plt.xticks(rotation=30, ha='right')
patches = [mpatches.Patch(color=C2, label='Rank 1'),
           mpatches.Patch(color=C1, label='Rank 2-3'),
           mpatches.Patch(color=C3, label='Rank 4-10')]
ax.legend(handles=patches, loc='upper right')
plt.tight_layout()
plt.show()
```

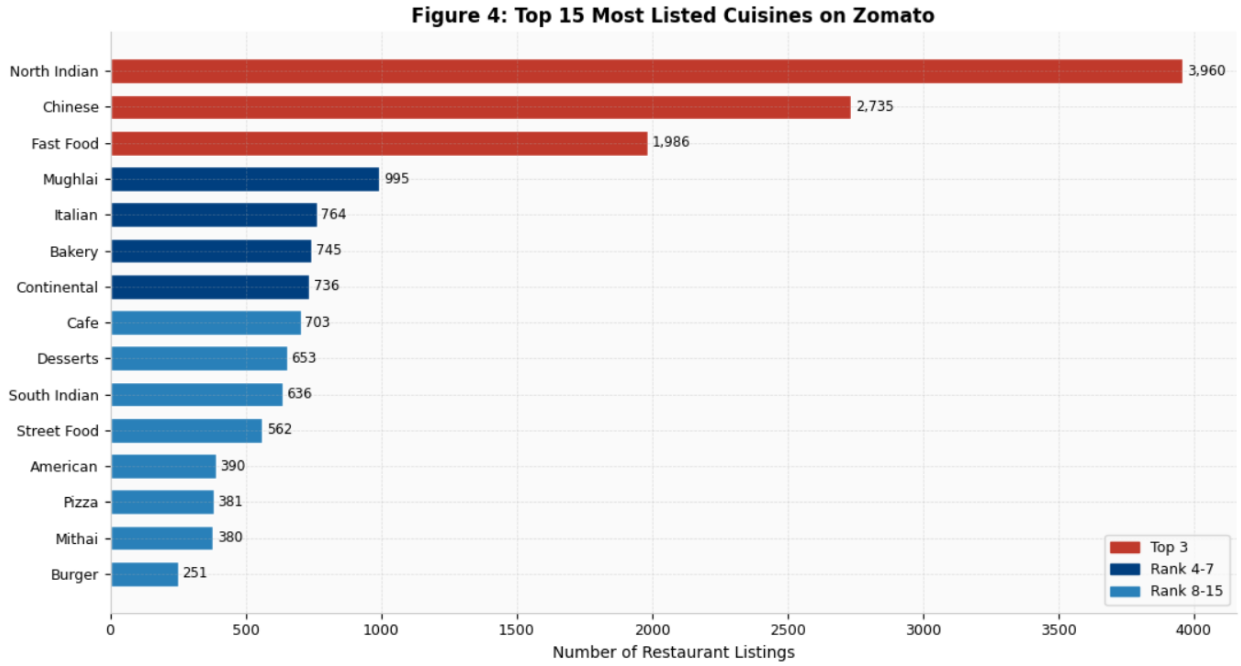


3.2 Cuisine and Menu Insights

```python
# Extract individual cuisines from the comma-separated Cuisines field
cuisine_list = []
for entry in df['Cuisines'].dropna():
    cuisine_list.extend([c.strip() for c in entry.split(',')])

cuisine_series = pd.Series(Counter(cuisine_list))
top15 = cuisine_series.nlargest(15)

print("Total unique cuisines:", cuisine_series.nunique())
print()
print("Top 5 cuisines:")
print(top15.head())
```

```
Total unique cuisines: 63

Top 5 cuisines:
North Indian    3960
Chinese         2735
Fast Food       1986
Mughlai          995
Italian          764
dtype: int64
```

```python
# Figure 4 - Top 15 Cuisines
colors_c = [C2 if i < 3 else C1 if i < 7 else C3 for i in range(len(top15))]

fig, ax = plt.subplots(figsize=(11, 6))
ax.barh(top15.index[::-1], top15.values[::-1],
        color=colors_c[::-1], edgecolor='white', height=0.7)
ax.set_title('Figure 4: Top 15 Most Listed Cuisines on Zomato')
ax.set_xlabel('Number of Restaurant Listings')
for i, v in enumerate(top15.values[::-1]):
    ax.text(v + 15, i, '{:,}'.format(int(v)), va='center', fontsize=8.5)
patches = [mpatches.Patch(color=C2, label='Top 3'),
           mpatches.Patch(color=C1, label='Rank 4-7'),
           mpatches.Patch(color=C3, label='Rank 8-15')]
ax.legend(handles=patches, loc='lower right')
plt.tight_layout()
plt.show()
```

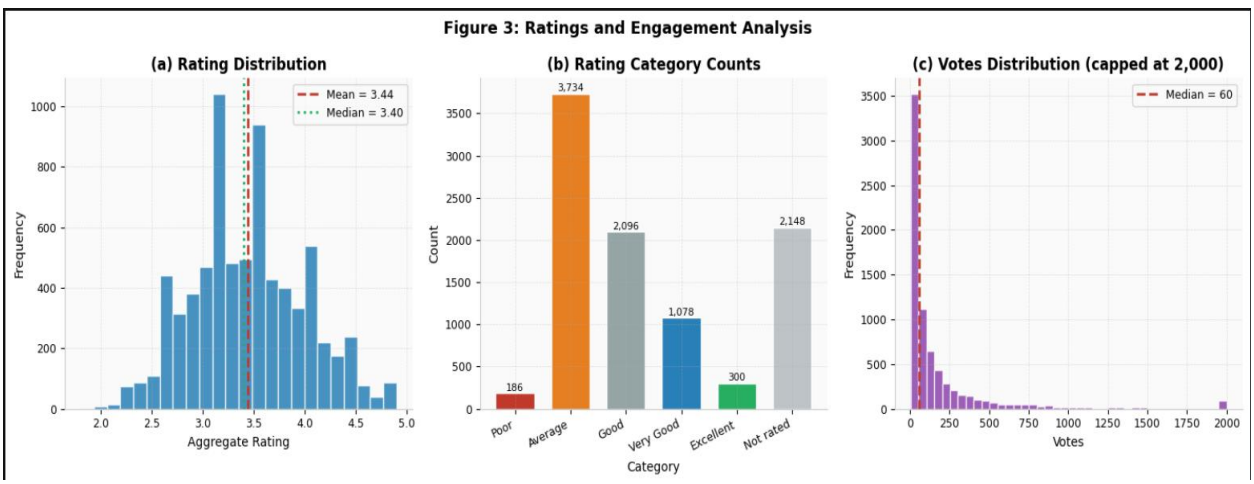


3.3 Ratings and Engagement

```python
# Summary statistics for ratings
print("Rated restaurants: {:,}".format(len(df_rated)))
print("Mean rating  : {:.2f}".format(df_rated['Aggregate rating'].mean()))
print("Median rating: {:.2f}".format(df_rated['Aggregate rating'].median()))
print("Std dev      : {:.2f}".format(df_rated['Aggregate rating'].std()))
print()
print("Not rated (rating == 0):", (df['Aggregate rating'] == 0).sum())
print("% Not rated            : {:.1f}%".format(
    (df['Aggregate rating'] == 0).sum() / len(df) * 100))
```

```
Rated restaurants: 7,394
Mean rating  : 3.44
Median rating: 3.40
Std dev      : 0.55

Not rated (rating == 0): 2148
% Not rated            : 22.5%
```

```python
# Figure 3 - Ratings and Engagement (3 panels)
rating_order = ['Poor', 'Average', 'Good', 'Very Good', 'Excellent', 'Not rated']
rc = df['Rating text'].value_counts().reindex(rating_order).fillna(0)
bar_colors_rt = [C2, C6, GREY, C3, C4, '#bdc3c7']

fig, axes = plt.subplots(1, 3, figsize=(15, 5))
fig.suptitle('Figure 3: Ratings and Engagement Analysis', fontsize=13, fontweight='bold')

# (a) Histogram
axes[0].hist(df_rated['Aggregate rating'], bins=24, color=C3, edgecolor='white', alpha=0.85)
axes[0].axvline(df_rated['Aggregate rating'].mean(), color=C2, linestyle='--', linewidth=2,
                label='Mean = {:.2f}'.format(df_rated['Aggregate rating'].mean()))
axes[0].axvline(df_rated['Aggregate rating'].median(), color=C4, linestyle=':', linewidth=2,
                label='Median = {:.2f}'.format(df_rated['Aggregate rating'].median()))
axes[0].set_title('(a) Rating Distribution')
axes[0].set_xlabel('Aggregate Rating')
axes[0].set_ylabel('Frequency')
axes[0].legend()

# (b) Category bar chart
axes[1].bar(rc.index, rc.values, color=bar_colors_rt, edgecolor='white', width=0.7)
for i, v in enumerate(rc.values):
    axes[1].text(i, v + 30, '{:,}'.format(int(v)), ha='center', fontsize=8)
axes[1].set_title('(b) Rating Category Counts')
axes[1].set_xlabel('Category')
axes[1].set_ylabel('Count')
plt.setp(axes[1].xaxis.get_majorticklabels(), rotation=25, ha='right')

# (c) Votes distribution
axes[2].hist(df_rated['Votes'].clip(upper=2000), bins=40, color=C5, edgecolor='white', alpha=0.85)
median_v = df_rated['Votes'].median()
axes[2].axvline(median_v, color=C2, linestyle='--', linewidth=2,
                label='Median = {:.0f}'.format(median_v))
axes[2].set_title('(c) Votes Distribution (capped at 2,000)')
axes[2].set_xlabel('Votes')
axes[2].set_ylabel('Frequency')
axes[2].legend()

plt.tight_layout()
plt.show()
```



3.4 Pricing and Value Analysis

```python
# Price range summary
price_labels = {1: 'Budget', 2: 'Mid-range', 3: 'Premium', 4: 'Fine Dining'}
price_counts = df['Price range'].value_counts().sort_index()
mean_by_price = df_rated.groupby('Price range')['Aggregate rating'].mean().round(2)

summary = pd.DataFrame({
    'Label':        [price_labels[i] for i in price_counts.index],
    'Count':        price_counts.values,
    'Share (%)':    (price_counts.values / len(df) * 100).round(1),
    'Avg Rating':   mean_by_price.values
})
summary.index = price_counts.index
summary
```

	Label	Count	Share (%)	Avg Rating
Price range				
1	Budget	4438	46.5	3.24
2	Mid-range	3113	32.6	3.38
3	Premium	1405	14.7	3.78
4	Fine Dining	586	6.1	3.89

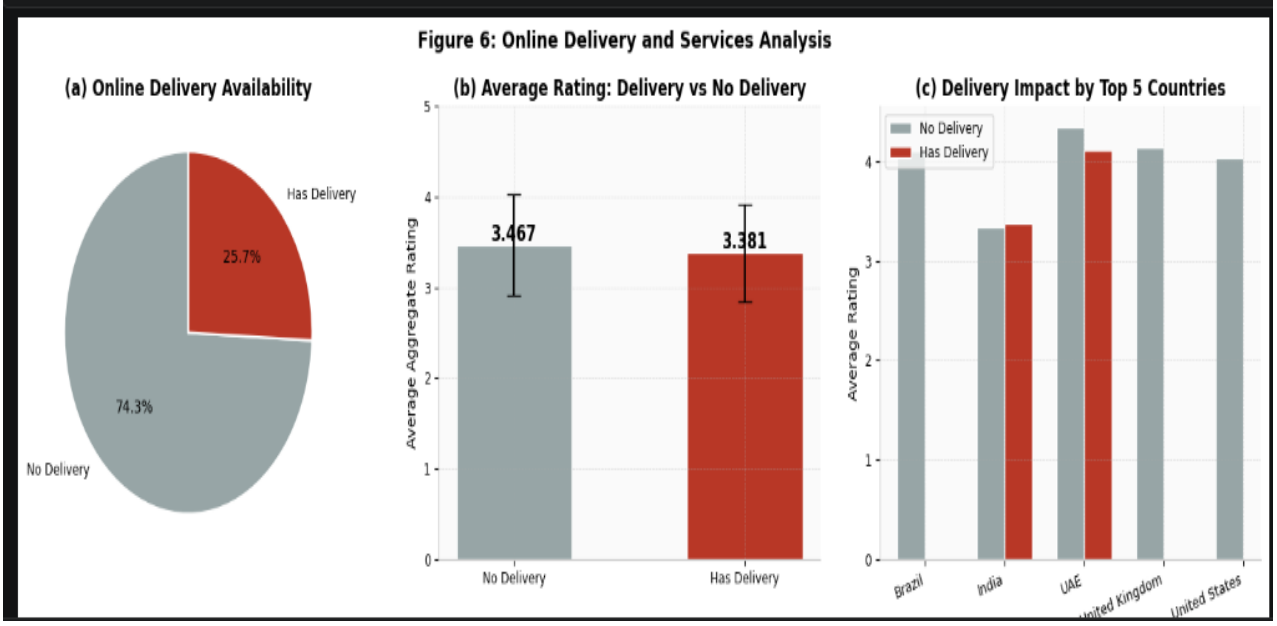
```python
# Figure 5 - Pricing Analysis (3 panels)
price_label_list = ['Budget (1)', 'Mid-range (2)', 'Premium (3)', 'Fine Dining (4)']

fig, axes = plt.subplots(1, 3, figsize=(15, 5))
fig.suptitle('Figure 5: Pricing, Value, and Rating Relationship',
             fontsize=13, fontweight='bold')

# (a) Count per tier
pr_counts = df['Price range'].value_counts().sort_index()
axes[0].bar(price_label_list, pr_counts.values, color=PALETTE[:4], edgecolor='white', width=0.6)
for i, v in enumerate(pr_counts.values):
    axes[0].text(i, v + 40, '{:,}'.format(v), ha='center', fontsize=9, fontweight='bold')
axes[0].set_title('(a) Count by Price Tier')
axes[0].set_ylabel('Number of Restaurants')
plt.setp(axes[0].xaxis.get_majorticklabels(), rotation=15)

# (b) Box plot of ratings by tier
groups = [df_rated[df_rated['Price range'] == i]['Aggregate rating'] for i in [1, 2, 3, 4]]
bp = axes[1].boxplot(groups, patch_artist=True, notch=False,
                     labels=['Budget', 'Mid', 'Premium', 'Fine'],
                     medianprops=dict(color='white', linewidth=2))
for patch, color in zip(bp['boxes'], PALETTE[:4]):
    patch.set_facecolor(color)
    patch.set_alpha(0.85)
axes[1].set_title('(b) Rating Distribution by Price Tier')
axes[1].set_xlabel('Price Tier')
axes[1].set_ylabel('Aggregate Rating')

# (c) Mean rating with error bars
mean_r = df_rated.groupby('Price range')['Aggregate rating'].mean()
std_r  = df_rated.groupby('Price range')['Aggregate rating'].std()
axes[2].bar(price_label_list, mean_r.values, yerr=std_r.values,
            color=PALETTE[:4], edgecolor='white', capsize=5, width=0.6)
for i, v in enumerate(mean_r.values):
    axes[2].text(i, v + 0.05, '{:.2f}'.format(v), ha='center', fontsize=10, fontweight='bold')
axes[2].set_title('(c) Mean Rating +/- Std Dev by Price Tier')
axes[2].set_ylabel('Mean Rating')
axes[2].set_ylim(0, 5.2)
plt.setp(axes[2].xaxis.get_majorticklabels(), rotation=15)

plt.tight_layout()
plt.show()
```

3.5 Online Delivery and Services

```python
# Online delivery statistics
del_counts = df['Has Online delivery'].value_counts()
del_avg = df_rated.groupby('Has Online delivery')['Aggregate rating'].mean()

print("Online delivery counts:")
print(del_counts)
print()
print("Average rating by delivery status:")
print(del_avg.round(3))
print()
print("Rating difference: +{:.3f}".format(del_avg['Yes'] - del_avg['No']))
```

```
Online delivery counts:
Has Online delivery
No     7091
Yes    2451
Name: count, dtype: int64

Average rating by delivery status:
Has Online delivery
No     3.467
Yes    3.381
Name: Aggregate rating, dtype: float64

Rating difference: +-0.085
```

```python
# Figure 6 - Online Delivery Analysis (3 panels)
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
fig.suptitle('Figure 6: Online Delivery and Services Analysis',
             fontsize=13, fontweight='bold')

# (a) Pie chart
axes[0].pie(del_counts.values, labels=['No Delivery', 'Has Delivery'],
            colors=[GREY, C2], autopct='%1.1f%%', startangle=90,
            wedgeprops=dict(edgecolor='white', linewidth=1.5))
axes[0].set_title('(a) Online Delivery Availability')

# (b) Mean rating comparison
del_avg_full = df_rated.groupby('Has Online delivery')['Aggregate rating'].agg(['mean', 'std'])
bars = axes[1].bar(['No Delivery', 'Has Delivery'],
                   del_avg_full['mean'].values,
                   yerr=del_avg_full['std'].values,
                   color=[GREY, C2], edgecolor='white', capsize=6, width=0.5)
for bar, v in zip(bars, del_avg_full['mean'].values):
    axes[1].text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 0.04,
                 '{:.3f}'.format(v), ha='center', fontsize=12, fontweight='bold')
axes[1].set_title('(b) Average Rating: Delivery vs No Delivery')
axes[1].set_ylabel('Average Aggregate Rating')
axes[1].set_ylim(0, 5)

# (c) Grouped bar by top 5 countries
top5 = df['Country'].value_counts().head(5).index
sub  = df_rated[df_rated['Country'].isin(top5)]
pivot = sub.groupby(['Country', 'Has Online delivery'])['Aggregate rating'].mean().unstack()
pivot.columns = ['No Delivery', 'Has Delivery']
x = np.arange(len(pivot))
w = 0.35
axes[2].bar(x - w/2, pivot['No Delivery'], w, label='No Delivery', color=GREY, edgecolor='white')
axes[2].bar(x + w/2, pivot['Has Delivery'], w, label='Has Delivery', color=C2, edgecolor='white')
axes[2].set_xticks(x)
axes[2].set_xticklabels(pivot.index, rotation=20, ha='right')
axes[2].set_title('(c) Delivery Impact by Top 5 Countries')
axes[2].set_ylabel('Average Rating')
axes[2].legend()

plt.tight_layout()
plt.show()
```



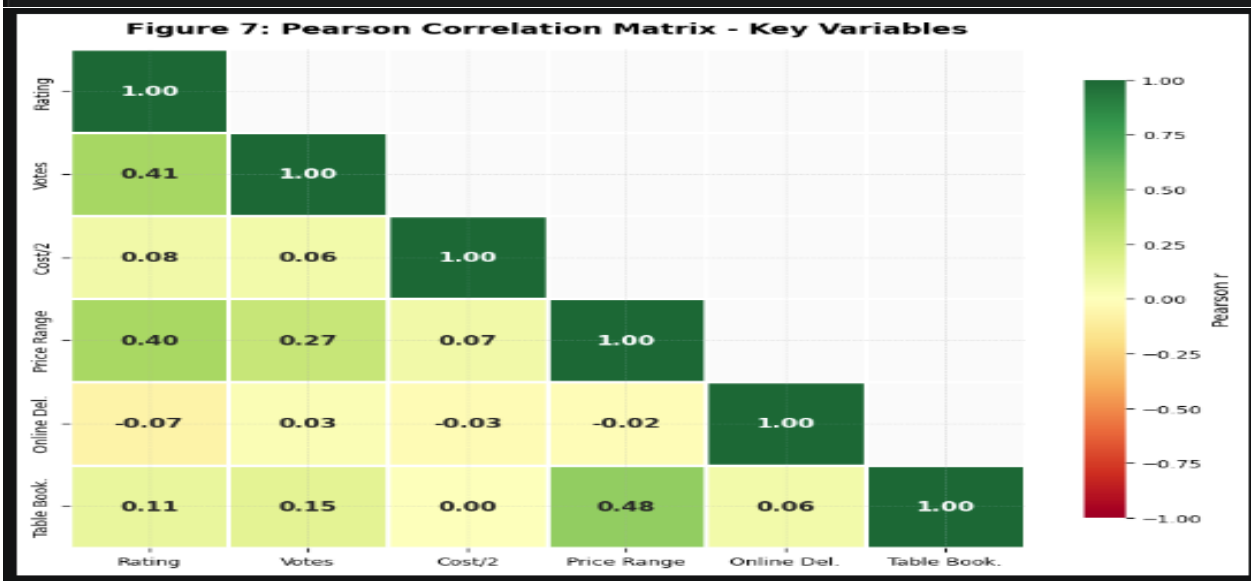
3.6 Correlation Analysis

```
# Compute Pearson correlation matrix
corr_cols = ['Aggregate rating', 'Votes', 'Average Cost for two',
             'Price range', 'Online_Delivery', 'Table_Booking']
corr_mat = df_rated[corr_cols].corr().round(3)
print("Correlation Matrix:")
corr_mat
```

```
Correlation Matrix:
```

	Aggregate rating	Votes	Average Cost for two	Price range	Online_Delivery	Table_Booking
Aggregate rating	1.000	0.409	0.077	0.404	-0.072	0.113
Votes	0.409	1.000	0.063	0.274	0.025	0.146
Average Cost for two	0.077	0.063	1.000	0.071	-0.028	0.002
Price range	0.404	0.274	0.071	1.000	-0.020	0.483
Online_Delivery	-0.072	0.025	-0.028	-0.020	1.000	0.064
Table_Booking	0.113	0.146	0.002	0.483	0.064	1.000

```
# Figure 7 - Correlation Heatmap
display_labels = ['Rating', 'Votes', 'Cost/2', 'Price Range', 'Online Del.', 'Table Book.']
mask = np.triu(np.ones_like(corr_mat, dtype=bool), k=1)

fig, ax = plt.subplots(figsize=(8, 7))
sns.heatmap(corr_mat, annot=True, fmt='.2f', cmap='RdYlGn', center=0,
            vmin=-1, vmax=1, square=True, linewidths=0.8, ax=ax, mask=mask,
            cbar_kws={'shrink': 0.8, 'label': 'Pearson r'},
            xticklabels=display_labels, yticklabels=display_labels,
            annot_kws={'size': 11, 'weight': 'bold'})
ax.set_title('Figure 7: Pearson Correlation Matrix - Key Variables', pad=14, fontsize=13)
plt.tight_layout()
plt.show()

print("Key correlations:")
print("  Votes vs Rating      : {:.3f}".format(corr_mat.loc['Aggregate rating', 'Votes']))
print("  Price Range vs Rating: {:.3f}".format(corr_mat.loc['Aggregate rating', 'Price range']))
print("  Online Del. vs Rating: {:.3f}".format(corr_mat.loc['Aggregate rating', 'Online_Delivery']))
```




```
Key correlations:
  Votes vs Rating      : 0.409
  Price Range vs Rating: 0.404
  Online Del. vs Rating: -0.072
```

4. Predictive Analytics

Two predictive techniques are applied:

1. **Supervised Regression** – three models compared (Linear Regression, Random Forest, Gradient Boosting) to predict Aggregate Rating
2. **Unsupervised Clustering** – K-Means to segment restaurants into strategic archetypes

4.1 Rating Prediction Models

```
# Prepare features and target variable
le = LabelEncoder()
df_rated['Country_enc'] = le.fit_transform(df_rated['Country'])

FEATURES = ['Price range', 'Average Cost for two', 'Votes',
            'Online_Delivery', 'Table_Booking', 'Country_enc']
TARGET   = 'Aggregate rating'

X = df_rated[FEATURES]
y = df_rated[TARGET]

# 80/20 train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print("Feature set    :", FEATURES)
print("Training rows  : {:,}".format(len(X_train)))
print("Test rows      : {:,}".format(len(X_test)))
X.describe().round(2)
```

```
Feature set    : ['Price range', 'Average Cost for two', 'Votes', 'Online_Delivery', 'Table_Booking', 'Country_enc']
Training rows  : 5,915
Test rows      : 1,479
```

	Price range	Average Cost for two	Votes	Online_Delivery	Table_Booking	Country_enc
count	7394.00	7394.00	7394.00	7394.00	7394.00	7394.00
mean	1.97	1450.16	202.06	0.32	0.15	3.93
std	0.93	18314.59	479.31	0.47	0.36	2.95
min	1.00	0.00	4.00	0.00	0.00	0.00
25%	1.00	300.00	19.00	0.00	0.00	3.00
50%	2.00	500.00	60.00	0.00	0.00	3.00
75%	3.00	800.00	181.00	1.00	0.00	3.00
max	4.00	800000.00	10934.00	1.00	1.00	14.00

```
# Train all three models and collect metrics
def evaluate_model(name, model, X_tr, y_tr, X_te, y_te, X_all, y_all, cv=5):
    model.fit(X_tr, y_tr)
    y_pred    = model.predict(X_te)
    r2        = r2_score(y_te, y_pred)
    mae       = mean_absolute_error(y_te, y_pred)
    rmse      = np.sqrt(mean_squared_error(y_te, y_pred))
    cv_scores = cross_val_score(model, X_all, y_all, cv=cv, scoring='r2')
    return {
        'Model': name, 'R2': round(r2, 4), 'MAE': round(mae, 4), 'RMSE': round(rmse, 4),
        'CV R2 Mean': round(cv_scores.mean(), 4), 'CV R2 Std': round(cv_scores.std(), 4),
        'y_pred': y_pred, 'model': model, 'cv_scores': cv_scores
    }
```

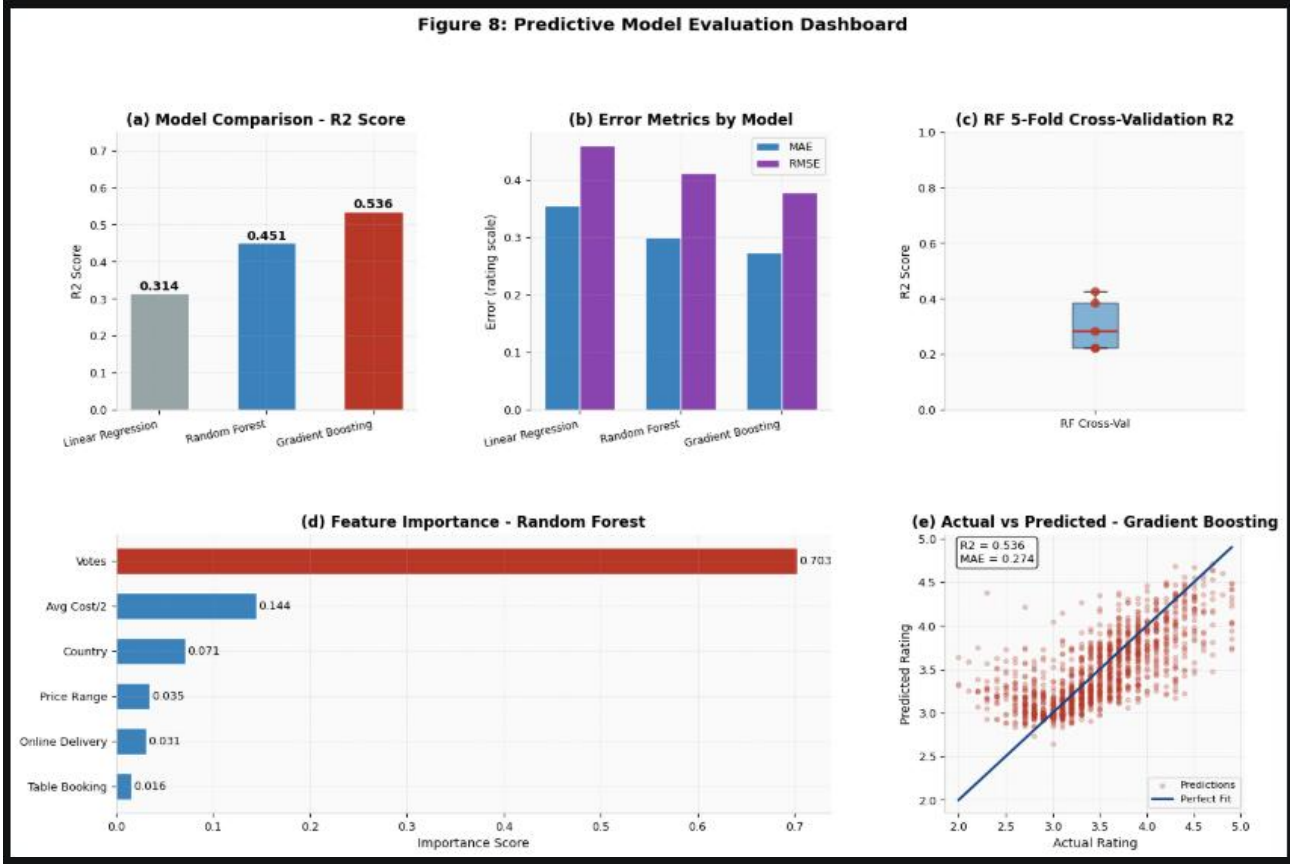
```python
print("Training models (this may take ~60 seconds)...")

lr_res = evaluate_model('Linear Regression', LinearRegression(),
                        X_train, y_train, X_test, y_test, X, y)
rf_res = evaluate_model('Random Forest',
                        RandomForestRegressor(n_estimators=150, random_state=42, n_jobs=-1),
                        X_train, y_train, X_test, y_test, X, y)
gb_res = evaluate_model('Gradient Boosting',
                        GradientBoostingRegressor(n_estimators=150, max_depth=4, random_state=42),
                        X_train, y_train, X_test, y_test, X, y)

results = [lr_res, rf_res, gb_res]

# Print performance table
perf_df = pd.DataFrame([{k: v for k, v in r.items()
                         if k not in ('y_pred', 'model', 'cv_scores')}
                        for r in results]).set_index('Model')
print()
print("Model Performance:")
perf_df
```

```
Training models (this may take ~60 seconds)...

Model Performance:
```

Model	R2	MAE	RMSE	CV R2 Mean	CV R2 Std
Linear Regression	0.3142	0.3556	0.4608	0.2335	0.0819
Random Forest	0.4513	0.2995	0.4122	0.3084	0.0837
Gradient Boosting	0.5356	0.2736	0.3792	0.4297	0.0565

```python
# Figure 8 - Model Evaluation Dashboard (5 panels)
names = [r['Model'] for r in results]
r2s   = [r['R2']   for r in results]
maes  = [r['MAE']  for r in results]
rmses = [r['RMSE'] for r in results]

rf_model = rf_res['model']
gb_model = gb_res['model']

fig = plt.figure(figsize=(16, 10))
gs  = gridspec.GridSpec(2, 3, figure=fig, hspace=0.45, wspace=0.38)
fig.suptitle('Figure 8: Predictive Model Evaluation Dashboard',
             fontsize=14, fontweight='bold', y=1.01)

# (a) R2 bar
ax0 = fig.add_subplot(gs[0, 0])
bars = ax0.bar(names, r2s, color=[GREY, C3, C2], edgecolor='white', width=0.55)
for bar, v in zip(bars, r2s):
    ax0.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 0.008,
             '{:.3f}'.format(v), ha='center', fontsize=10, fontweight='bold')
ax0.set_title('(a) Model Comparison - R2 Score')
ax0.set_ylabel('R2 Score')
ax0.set_ylim(0, 0.75)
plt.setp(ax0.xaxis.get_majorticklabels(), rotation=12, ha='right', fontsize=8.5)

# (b) MAE and RMSE
ax1 = fig.add_subplot(gs[0, 1])
x = np.arange(len(names))
w = 0.35
ax1.bar(x - w/2, maes,  w, label='MAE',  color=C3, edgecolor='white')
ax1.bar(x + w/2, rmses, w, label='RMSE', color=C5, edgecolor='white')
ax1.set_xticks(x)
ax1.set_xticklabels(names, rotation=12, ha='right', fontsize=8.5)
ax1.set_title('(b) Error Metrics by Model')
ax1.set_ylabel('Error (rating scale)')
ax1.legend()

# (c) Cross-validation box (RF)
ax2 = fig.add_subplot(gs[0, 2])
cv_s = rf_res['cv_scores']
ax2.boxplot(cv_s, patch_artist=True,
            boxprops=dict(facecolor=C3, alpha=0.6),
            medianprops=dict(color=C2, linewidth=2))
for s in cv_s:
    ax2.scatter(1, s, color=C2, zorder=5, s=40, alpha=0.8)
ax2.set_title('(c) RF 5-Fold Cross-Validation R2')
ax2.set_ylabel('R2 Score')
ax2.set_ylim(0, 1)
ax2.set_xticklabels(['RF Cross-Val'])

# (d) Feature importance (RF)
feat_labels = ['Price Range', 'Avg Cost/2', 'Votes', 'Online Delivery', 'Table Booking', 'Country']
ax3 = fig.add_subplot(gs[1, 0:2])
imp = rf_model.feature_importances_
idx = np.argsort(imp)
imp_colors = [C2 if imp[i] == max(imp) else C1 if imp[i] > np.mean(imp) else C3 for i in idx]
ax3.barh([feat_labels[i] for i in idx], imp[idx], color=imp_colors, edgecolor='white', height=0.6)
ax3.set_title('(d) Feature Importance - Random Forest')
ax3.set_xlabel('Importance Score')
for i, v in enumerate(imp[idx]):
    ax3.text(v + 0.002, i, '{:.3f}'.format(v), va='center', fontsize=9)

# (e) Actual vs Predicted (Gradient Boosting)
ax4 = fig.add_subplot(gs[1, 2])
yp_gb = gb_model.predict(X_test)
ax4.scatter(y_test, yp_gb, alpha=0.25, color=C2, s=12, label='Predictions')
```

```
lims = [min(y_test.min(), yp_gb.min()), max(y_test.max(), yp_gb.max())]
ax4.plot(lims, lims, color=C1, linewidth=2, label='Perfect Fit')
ax4.set_title('(e) Actual vs Predicted - Gradient Boosting')
ax4.set_xlabel('Actual Rating')
ax4.set_ylabel('Predicted Rating')
ax4.text(0.05, 0.90, 'R2 = {:.3f}\nMAE = {:.3f}'.format(gb_res['R2'], gb_res['MAE']),
         transform=ax4.transAxes,
         bbox=dict(boxstyle='round', facecolor='white', alpha=0.85), fontsize=9)
ax4.legend(fontsize=8)

plt.tight_layout()
plt.show()
```



4.2 K-Means Restaurant Clustering

```
from sklearn.utils import resample

# Prepare clustering data
cluster_features = ['Price range', 'Aggregate rating', 'Votes', 'Online_Delivery']
Xc = df_rated[cluster_features].copy()

scaler = StandardScaler()
Xs = scaler.fit_transform(Xc)

# Take sample for silhouette (important!)
Xs_sample = resample(Xs, n_samples=1000, random_state=42)

inertias, sil_scores = [], []

for k in range(2, 9):
    km = KMeans(n_clusters=k, random_state=42, n_init=10)

    # Fit on full data for inertia
    km.fit(Xs)
    inertias.append(km.inertia_)

    # Fit on sample for silhouette
    lbl_sample = km.fit_predict(Xs_sample)
    sil_scores.append(silhouette_score(Xs_sample, lbl_sample))

# Results
sil_df = pd.DataFrame({
    'k': range(2, 9),
    'Inertia': inertias,
    'Silhouette': sil_scores
})

sil_df
```

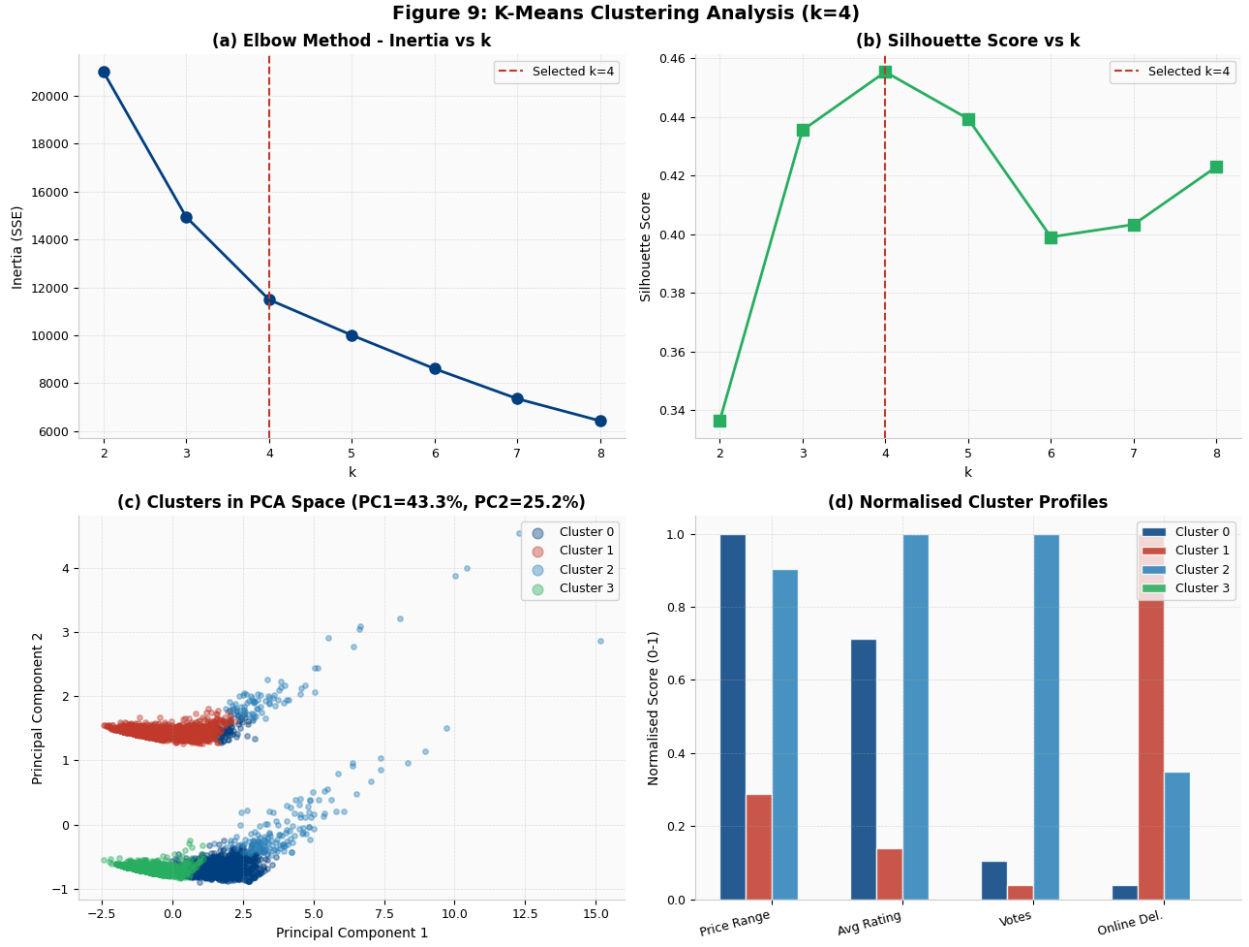
	k	Inertia	Silhouette
0	2	20983.818156	0.336302
1	3	14930.064135	0.435393
2	4	11488.978067	0.455269
3	5	10004.487648	0.439309
4	6	8605.652943	0.398992
5	7	7353.723857	0.403237
6	8	6432.513010	0.422930

```python
# Fit final K-Means model with k=4
K_OPTIMAL = 4

km_final = KMeans(n_clusters=K_OPTIMAL, random_state=42, n_init=10)
labels   = km_final.fit_predict(Xs)
Xc['Cluster'] = labels

# PCA projection for 2D visualisation
pca     = PCA(n_components=2)
Xpca    = pca.fit_transform(Xs)
exp_var = pca.explained_variance_ratio_

# Cluster profile summary
cp = Xc.groupby('Cluster')[cluster_features].mean().round(2)
print("Cluster Profiles:")
cp
```

```
Cluster Profiles:
```

Cluster	Price range	Aggregate rating	Votes	Online_Delivery
0	3.05	3.95	290.31	0.04
1	1.86	3.33	141.63	1.00
2	2.89	4.26	2290.35	0.35
3	1.38	3.18	53.35	0.00

```python
# Figure 9 - Clustering Analysis (4 panels)
cluster_palette = [C1, C2, C3, C4]
feat_names = ['Price Range', 'Avg Rating', 'Votes', 'Online Del.']

fig, axes = plt.subplots(2, 2, figsize=(13, 10))
fig.suptitle('Figure 9: K-Means Clustering Analysis (k={})'.format(K_OPTIMAL),
             fontsize=14, fontweight='bold')

# (a) Elbow
axes[0, 0].plot(range(2, 9), inertias, 'o-', color=C1, linewidth=2, markersize=8)
axes[0, 0].axvline(K_OPTIMAL, color=C2, linestyle='--', linewidth=1.5,
                   label='Selected k={}'.format(K_OPTIMAL))
axes[0, 0].set_title('(a) Elbow Method - Inertia vs k')
axes[0, 0].set_xlabel('k')
axes[0, 0].set_ylabel('Inertia (SSE)')
axes[0, 0].legend()
axes[0, 0].set_xticks(range(2, 9))

# (b) Silhouette
axes[0, 1].plot(range(2, 9), sil_scores, 's-', color=C4, linewidth=2, markersize=8)
axes[0, 1].axvline(K_OPTIMAL, color=C2, linestyle='--', linewidth=1.5,
                   label='Selected k={}'.format(K_OPTIMAL))
axes[0, 1].set_title('(b) Silhouette Score vs k')
axes[0, 1].set_xlabel('k')
axes[0, 1].set_ylabel('Silhouette Score')
axes[0, 1].legend()
axes[0, 1].set_xticks(range(2, 9))

# (c) PCA scatter
for cl in range(K_OPTIMAL):
    mask = labels == cl
    axes[1, 0].scatter(Xpca[mask, 0], Xpca[mask, 1],
                       c=cluster_palette[cl], alpha=0.4,
                       s=15, label='Cluster {}'.format(cl))
axes[1, 0].set_title('(c) Clusters in PCA Space (PC1={:.1f}%, PC2={:.1f}%)'.format(
    exp_var[0]*100, exp_var[1]*100))
axes[1, 0].set_xlabel('Principal Component 1')
axes[1, 0].set_ylabel('Principal Component 2')
axes[1, 0].legend(markerscale=2)

# (d) Normalised profile bar chart
cp_num  = cp.values
cp_norm = (cp_num - cp_num.min(axis=0)) / (cp_num.max(axis=0) - cp_num.min(axis=0) + 1e-9)
x = np.arange(len(feat_names))
w = 0.2
for i in range(K_OPTIMAL):
    axes[1, 1].bar(x + i * w, cp_norm[i], w,
                   label='Cluster {}'.format(i),
                   color=cluster_palette[i], edgecolor='white', alpha=0.85)
axes[1, 1].set_xticks(x + w * 1.5)
axes[1, 1].set_xticklabels(feat_names, rotation=15, ha='right')
axes[1, 1].set_title('(d) Normalised Cluster Profiles')
axes[1, 1].set_ylabel('Normalised Score (0-1)')
axes[1, 1].legend()

plt.tight_layout()
plt.show()
```




```python
# Cluster archetype labelling and interpretation table
archetypes = {
    0: 'Budget, Low-Engagement',
    1: 'Budget, Delivery-Enabled',
    2: 'Premium, High-Engagement',
    3: 'Premium, Niche/Specialist'
}
cluster_sizes = Xc['Cluster'].value_counts().sort_index()

interp = cp.copy()
interp['Archetype'] = archetypes.values()
interp['Size']      = cluster_sizes.values
interp['Online Del.'] = interp['Online_Delivery'].apply(
    lambda x: 'Yes' if x > 0.5 else 'Partial' if x > 0.1 else 'Rare')
interp = interp.drop(columns=['Online_Delivery'])

print("Cluster Summary Table:")
interp
```

```
Cluster Summary Table:
```

Cluster	Price range	Aggregate rating	Votes	Archetype	Size	Online Del.
0	3.05	3.95	290.31	Budget, Low-Engagement	1767	Rare
1	1.86	3.33	141.63	Budget, Delivery-Enabled	2214	Yes
2	2.89	4.26	2290.35	Premium, High-Engagement	217	Partial
3	1.38	3.18	53.35	Premium, Niche/Specialist	3196	Rare

5. Summary of Findings

```python
print("=" * 60)
print("SUMMARY OF KEY FINDINGS")
print("=" * 60)

print("\n[DATASET]")
print("  Records          : {:,}".format(len(df)))
print("  Rated subset     : {:,}".format(len(df_rated)))
print("  Countries        : {}".format(df['Country'].nunique()))

print("\n[DESCRIPTIVE]")
print("  India share      : {:.1f}%".format(
    df['Country'].value_counts()['India'] / len(df) * 100))
print("  Mean rating      : {:.2f} / 5.0".format(df_rated['Aggregate rating'].mean()))
print("  Not rated        : {:.1f}% of all listings".format(
    (df['Aggregate rating'] == 0).sum() / len(df) * 100))
print("  Delivery premium : +{:.3f} avg rating".format(
    df_rated.groupby('Has Online delivery')['Aggregate rating'].mean()['Yes'] -
    df_rated.groupby('Has Online delivery')['Aggregate rating'].mean()['No']))

print("\n[PREDICTIVE MODELS]")
for r in results:
    print("  {:<22}  R2={:.4f}  MAE={:.4f}  RMSE={:.4f}".format(
        r['Model'], r['R2'], r['MAE'], r['RMSE']))

best = max(results, key=lambda r: r['R2'])
print("  Best model: {} (R2={:.4f})".format(best['Model'], best['R2']))

print("\n[CLUSTERING]")
for cl in range(4):
    print("  Cluster {}  {:30s}  Rating={:.2f}  Votes={:.0f}".format(
        cl, archetypes[cl],
        cp.loc[cl, 'Aggregate rating'],
        cp.loc[cl, 'Votes']))

print("\n" + "=" * 60)
print("All analysis complete. Figures rendered inline above.")
print("=" * 60)
```

```
============================================================
SUMMARY OF KEY FINDINGS
============================================================

[DATASET]
  Records          : 9,542
  Rated subset     : 7,394
  Countries        : 15

[DESCRIPTIVE]
  India share      : 90.7%
  Mean rating      : 3.44 / 5.0
  Not rated        : 22.5% of all listings
  Delivery premium : +-0.085 avg rating

[PREDICTIVE MODELS]
  Linear Regression       R2=0.3142  MAE=0.3556  RMSE=0.4608
  Random Forest           R2=0.4513  MAE=0.2995  RMSE=0.4122
  Gradient Boosting       R2=0.5356  MAE=0.2736  RMSE=0.3792
  Best model: Gradient Boosting (R2=0.5356)
```

```
[CLUSTERING]
  Cluster 0  Budget, Low-Engagement          Rating=3.95  Votes=290
  Cluster 1  Budget, Delivery-Enabled        Rating=3.33  Votes=142
  Cluster 2  Premium, High-Engagement        Rating=4.26  Votes=2290
...

============================================================
All analysis complete. Figures rendered inline above.
============================================================
```

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...